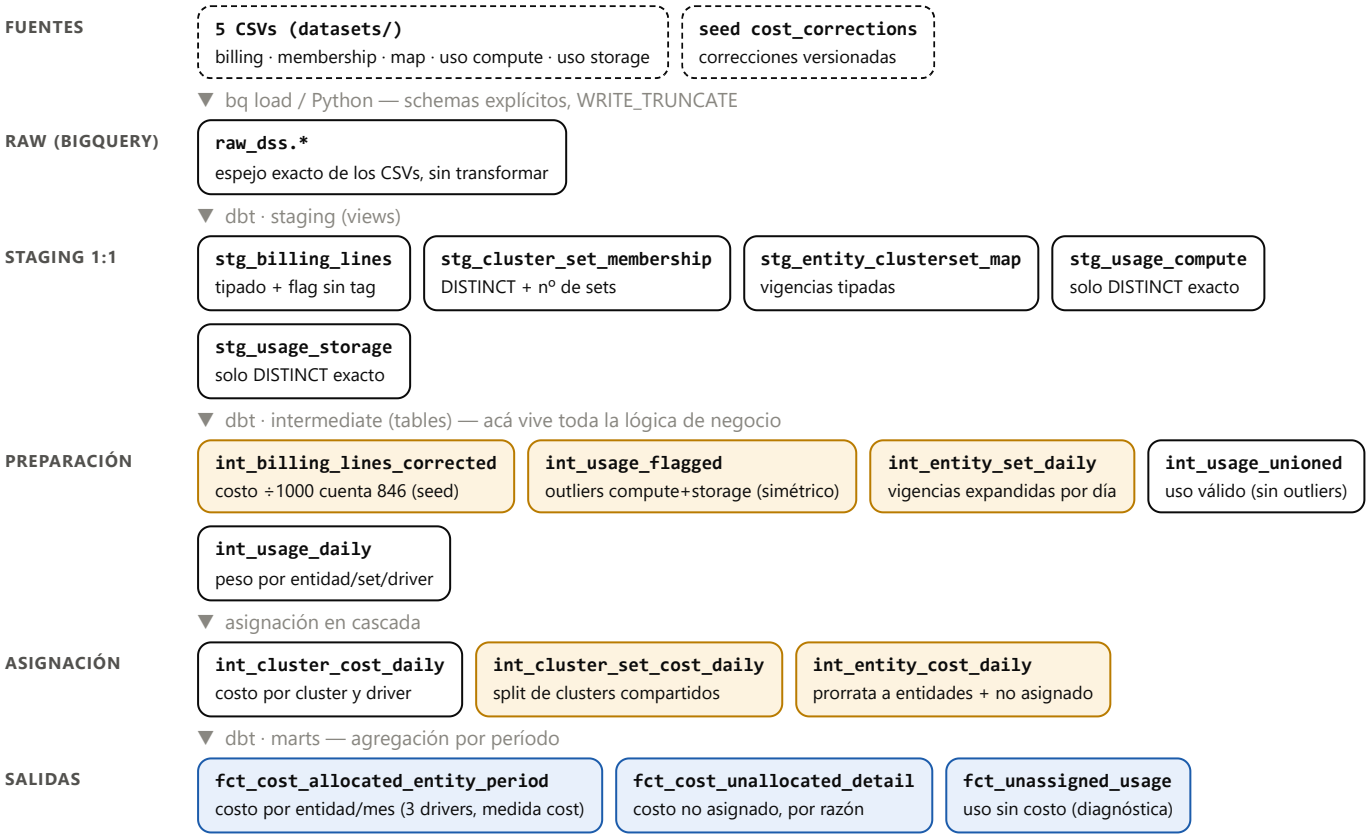
Pipeline de Cost Allocation `dss` — resumen de alto nivel

Cómo fluye el dato desde el billing crudo hasta el costo por aplicación · BigQuery + dbt + Airflow · Leyenda: REGLA regla de negocio aplicada · DECISIÓN decisión de diseño/método · TEST control automático

1 · El flujo completo de un vistazo



Orquestación: DAG de Airflow `load_raw_to_bq ≫ dbt_build ≫ notify_summary` (trigger manual, reproceso por mes vía parámetro). `dbt build` corre modelos y tests juntos: si un control crítico falla, el pipeline falla.

2 · Paso a paso: qué transforma cada capa

Carga RAW — `scripts/load_raw_to_bq.{sh,py}`

Copia los 5 CSVs a BigQuery tal cual vienen (idempotente: cada corrida reemplaza la tabla). DECISIÓN Schemas explícitos compartidos (`scripts/bq_schemas/*.json`), nunca autodetect: los `account_id` traen cero inicial (`088420000317`) y la inferencia los convertiría a números, destruyéndolos. IDs siempre STRING.

Staging — espejo tipado, sin lógica de negocio

Cinco vistas 1:1 con la fuente: renombran, tipan y colapsan copias exactas. DECISIÓN El layering es estricto: staging no joinea fuentes ni aplica reglas — toda decisión de negocio vive en intermediate.

DECISIÓN **Política de duplicados uniforme:** antes de deduplicar se pregunta qué *es* una fila. **(A)** copia **exacta** de una relación/medición → `DISTINCT` mecánico, sin pérdida. **(B)** misma clave con **valores en conflicto** → nunca se resuelve con un `min()` / `row_number()` arbitrario: pasa a una regla explícita o rompe un test. Staging solo hace (A).

- `stg_billing_lines` — tipado + `is_untagged` (línea sin cluster = overhead futuro). El costo acá es el *original* del feed.
- `stg_cluster_set_membership` — REGLA caso (A): `DISTINCT` elimina el duplicado exacto cl-088 (relación pura, repetirla no agrega información) y marca los 6 clusters en 2 sets (`sets_per_cluster`).
- `stg_usage_compute` / `stg_usage_storage` — REGLA caso (A) **simétrico** en ambos: solo `DISTINCT` (colapsa el 2025-08-20 de compute, 100% idéntico). Sin dedup por clave: un conflicto de valores (hoy solo el par outlier de storage) llega intacto a la regla de outlier.

Intermediate · preparación de insumos

- `int_billing_lines_corrected` — REGLA aplica la corrección de la cuenta 846 (facturaba ×1000: ~$1,5M/mes → $1.500): join a la seed por cuenta + vigencia; sin match, factor 1.0. DECISIÓN la corrección es *dato versionado* (factor, vigencia, autoría, razón), no código; `cost_original` e `is_corrected` quedan para auditoría. Todo lo downstream lee costo **solo** de acá.
- `int_usage_flagged` — union de compute + storage con su driver + detección de outlier **simétrica por entidad × driver**. REGLA outlier = uso > 5× la mediana móvil de 14 días Y existe otra fila normal esa misma fecha/driver. DECISIÓN la condición dual excluye del peso **solo** ante evidencia positiva de registro corrupto; un valor alto **aislado** no se excluye (un pico real se cobra a quien lo generó) — lo lista un test warn. El outlier se conserva flageado como evidencia, no se corrige el valor.
- `int_entity_set_daily` — expande el mapa entidad↔set a una fila por día. REGLA `valid_to` exclusivo (verificado: app-loyalty-01 migra de set el 2025-09-01 sin doble conteo). DECISIÓN vigencias solapadas son ambas válidas (app-fraud-04 pesa en sus 2 sets durante jun–jul).
- `int_usage_unioned` — `int_usage_flagged` sin outliers: la **única definición** de qué uso cuenta (ambos drivers). La consumen el peso de asignación y el reporte de uso huérfano, que así no pueden divergir. Su unicidad estricta post-exclusión rompe el build ante cualquier conflicto de clave no resuelto.
- `int_usage_daily` — cruza el uso válido con los sets vigentes → peso por (día, entidad, set, driver). DECISIÓN el uso de las 5 entidades sin mapeo no entra al peso: se reporta aparte como evidencia (`orphan_entity_usage`), sin recibir costo.

Intermediate · asignación en cascada (cluster → set → entidad)

- `int_cluster_cost_daily` — agrega el costo corregido por (día, cluster, driver, cuenta). REGLA mapeo `product_code → cost_driver` : EC2=compute, S3=storage. Las líneas sin tag (DataTransfer, CloudWatch, GuardDuty, Support) quedan fuera: son overhead y se prorratean recién en el mart. DECISIÓN atribuibilidad (tag) y naturaleza (producto) son dimensiones ortogonales que hoy coinciden 100%; un test las guarda para que un feed futuro desalineado no misroutee en silencio.
- `int_cluster_set_cost_daily` — reparte el costo del cluster a sus sets: 1 set → 100%; REGLA 2 sets → proporcional al uso de cada set ese día para ese driver (**EC2 se reparte por vCPU-hours y S3 por GB — nunca se mezclan unidades**); DECISIÓN si ningún set tiene uso, fallback 50/50 (parametrizado; hoy nunca ocurre). Cluster sin set → pasa entero a no asignado (`unmapped_cluster`).
- `int_entity_cost_daily` — reparte el costo del set entre sus entidades vigentes, proporcional al uso del driver. Casos borde, todos explícitos:
  - REGLA uso total > 0 → prorrata normal (`usage_proportional`).
  - DECISIÓN uso 0 (capacidad reservada ociosa) → **even-split entre las entidades vigentes**, flageado: `sole_member_no_usage` si el pool tiene 1 entidad (cuenta 512 en septiembre; también abril de app-buyer-07 y app-payments-05), `even_split_no_usage` si tiene N>1 (hoy no ocurre). Unificar N=1 y N>1 elimina la discontinuidad indefendible (cobrar 100% a una entidad sola pero $0 a un pool de dos). Nunca se divide por cero; nunca es silencioso.
  - REGLA set sin entidades vigentes → no asignado (`orphan_cluster_set` : cs-010/035/040 + el hallazgo cs-058, que factura 15 días antes de la vigencia de su única app).

Marts — las tablas de salida (2 principales + 1 diagnóstica)

- `fct_cost_allocated_entity_period` — grano: mes × entidad × driver × método. DECISIÓN **formato largo (tidy)**: una sola medida aditiva `cost` y `cost_driver ∈ {compute, storage, overhead}` . El overhead sin tag se prorratea por costo directo y sale como filas `cost_driver='overhead'` (no como columna solapada): así es graficable como un slice más y se elimina el riesgo de doble conteo. Directo = filtrar compute/storage; fully loaded = todo. DECISIÓN en producción es incremental por partición mensual; en sandbox corre como tabla (el sandbox expira particiones >60 días — verificado, documentado en el modelo).
- `fct_cost_unallocated_detail` — solo **costo** (USD), grano mes × razón: `unmapped_cluster` , `orphan_cluster_set` . Medida única que participa de la reconciliación sin filtros. El overhead ya no está acá (driver asignado).
- `fct_unassigned_usage` — DECISIÓN tabla diagnóstica separada: **uso** sin costo (vCPU-h/GB) por entidad × driver, razones `orphan_entity_usage` y `excluded_outlier_usage` . Costo y uso sin asignar nunca coexisten en una fila — mezclarlos era meter dos facts (medida, unidad, grano y rol distintos) en una tabla. El filtro `cost is not null` que la reconciliación necesitaba era el síntoma; separando, desaparece.

Controles automáticos (corren en cada `dbt build`)

- TEST `assert_reconciliation` — **el crítico**: por mes, asignado (compute + storage + overhead, contado una sola vez) + no asignado = facturado corregido, con tolerancia $0,01. Garantiza que ningún dólar se pierde ni se duplica.
- TEST `assert_unit_cost_in_range` — costo unitario implícito por cuenta/driver dentro de [0.1×, 10×] de la mediana global: detectaría un futuro "caso 846" en datos nuevos.
- TEST `assert_single_row_usage_spikes` (warn) — anomalías de uso de fila única que la regla de outlier no excluye: insumo de preguntas al dueño (¿pico real o error de unidad?).
- TEST `assert_no_usage_allocations_flagged` (warn) — lista toda asignación sin uso (`sole_member_no_usage` + `even_split_no_usage`): asignado pero nunca silencioso.
- TEST `assert_tag_product_alignment` (error) — falla si un EC2/S3 llega sin tag o un servicio de otra clase con tag: protege el supuesto del modelo de overhead ante datos nuevos.
- TEST ~100 tests genéricos: unicidad de claves por capa (la estricta post-dedup de uso, en `int_usage_unioned` , cubre ambos drivers), valores aceptados en métodos/razones, integridad billing→membership (warn: los 4 clusters sin mapeo son conocidos; alerta si crece).

3 · Mapa de decisiones → dónde viven en el código

Regla / decisión	Qué hace	Dónde
IDs como STRING, schemas explícitos	Preserva ceros iniciales de `account_id`	`scripts/bq_schemas/*.json`
Corrección ×1000 cuenta 846	Factor 0.001 con vigencia y autoría, auditable	`seeds/cost_corrections.csv` → `int_billing_lines_corrected`
Política de duplicados uniforme	DISTINCT solo para copias exactas; conflicto → regla o test, nunca `min()`	`stg_usage_compute` / `stg_usage_storage`
Outlier simétrico, excluido no corregido	>5× mediana móvil 14d + fila normal misma fecha, por driver	`int_usage_flagged` (vars en `dbt_project.yml`)
Vigencias: fin exclusivo, solapes válidos	Migraciones sin doble conteo; fraud-04 pesa en 2 sets	`int_entity_set_daily`
Split de clusters compartidos por uso	Proporcional al uso del set por día/driver; fallback 50/50	`int_cluster_set_cost_daily`
Prorrata por uso medido, por driver	EC2 por vCPU-hours, S3 por GB, por día	`int_entity_cost_daily`
Capacidad ociosa (sin uso): even-split flageado	`sole_member` (N=1) / `even_split` (N>1), sin discontinuidad	`int_entity_cost_daily` + test warn
Supuesto del overhead (tag ⇔ producto)	Rompe el build si un feed lo desalinea, no misroutea	`tests/assert_tag_product_alignment.sql`
Overhead como driver asignado (formato largo)	Una medida `cost`, overhead graficable como slice, sin doble conteo	`fct_cost_allocated_entity_period`
Costo y uso sin asignar en tablas separadas	Una fact = una medida/unidad/grano; sin mezclar dólares y uso	`fct_cost_unallocated_detail` · `fct_unassigned_usage`
Reconciliación al centavo	Asignado + no asignado = facturado, por mes	`tests/assert_reconciliation.sql`

Pipeline dss-cost-allocation · dbt + BigQuery + Airflow · período 2025-04 a 2025-09 · resultado: $249k/mes facturado corregido, 92,3% asignado a 120 entidades, $19k/mes no asignado con causa identificada. Fuente HTML versionada junto al PDF (docs/resumen_pipeline.html). Detalle ejecutivo en docs/propuesta.html.